FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Diaz, James Daniel M.
Section:	AN42
Professor:	Crisola G. Tan
Date:	October 4, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.

4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()
```

```
# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:

- How agents spread across the grid over time
- Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
  - **More agents** (e.g., 10 or 20)
  - **Smaller/larger grid sizes**
  - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)

Lab Task 1:

```
#code that shows the path

import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

```
# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))
```

```
# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
```

```
        self.position = (x, y)
        self.path.append(self.position)
```

```
# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)
```

```
        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

            agent.path = [agent.position]
```

```
        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)
```

```
    def step(self):
        for agent in self.agents:
            agent.step()
```

```
# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()
```

```
# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)
```

```
scat = ax.scatter([], [], s=200)  # s = size of agents
```

```
lines = [
    ax.plot([], [], linewidth=1)[0]
    for agent in model.agents
]
```

```
def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
```

```
    for agent, line in zip(model.agents, lines):
      path_x = [pos[0] for pos in agent.path]
      path_y = [pos[1] for pos in agent.path]
```

```
      line.set_data(path_x, path_y)
```

```
    return scat, *lines
```

```
ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True,
repeat=False)
plt.show()
```

Lab Task 2:

```
#code that makes them avoid each other


import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

```
# -------- Get user input --------
```

```
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))
min_distance = float(input("Enter minimum distance (e.g., 2): "))
```

```
# -------- Agent Definition --------
```

```
class RandomWalker(ap.Agent):
```

```
    def setup(self):
        self.position = (0, 0)
        self.path = []
```

```
    def propose_move(self, occupied):
        x, y = self.position
        width, height = self.model.p.grid_size
        radius = self.model.p.min_distance
```

```
        directions = [
            (1, 0), (-1, 0),
            (0, 1), (0, -1),
            (0, 0)
        ]
        random.shuffle(directions)
```

```
        candidates = []
```

```
        for dx, dy in directions:
            nx = max(0, min(width - 1, x + dx))
            ny = max(0, min(height - 1, y + dy))
```

```
            # Skip positions outside the permitted distance
            safe = True
            nearest_distance = float("inf")
```

```
            for ox, oy in occupied:
                if (ox, oy) == self.position:
                    continue
```

```
                distance_squared = (
                    (nx - ox) ** 2 + (ny - oy) ** 2
                )
```

```
                nearest_distance = min(
                    nearest_distance, distance_squared ** 0.5
                )
```

```
                if distance_squared < radius ** 2:
                    safe = False
                    break
```

```
            if safe:
                # Prefer moving away from nearby agents.
                # Random tie-breaking preserves random movement.
                candidates.append((nearest_distance, (nx, ny)))

        if not candidates:
            return self.position

        # Prefer the destination with the greatest separation.
        best_distance = max(score for score, pos in candidates)
        best_positions = [
            pos for score, pos in candidates
            if score == best_distance
        ]

        return random.choice(best_positions)


# -------- Model Definition --------

class RandomWalkModel(ap.Model):

    def setup(self):
        self.agents = ap.AgentList(
            self, self.p.agents, RandomWalker
        )

        width, height = self.p.grid_size

        # Initially place agents in distinct cells.
        cells = [
            (x, y)
            for x in range(width)
            for y in range(height)
        ]

        if self.p.agents > len(cells):
            raise ValueError(
                "There cannot be more agents than grid cells."
            )

        random.shuffle(cells)
```

```
        for agent, position in zip(self.agents, cells):
            agent.position = position
            agent.path = [position]
```

```
        # Shared occupancy record.
        self.grid = ap.Grid(
            self, self.p.grid_size, torus=False
        )
        self.grid.add_agents(self.agents)
```

```
    def step(self):
        # Snapshot of positions at the start of the step.
        occupied = [agent.position for agent in self.agents]
```

```
        # Every agent proposes before any agent moves.
        proposals = {
            agent: agent.propose_move(occupied)
            for agent in self.agents
        }
```

```
        # Resolve multiple agents selecting the same destination.
        destination_groups = {}
```

```
        for agent, destination in proposals.items():
            destination_groups.setdefault(
                destination, []
            ).append(agent)
```

```
        approved = {}
```

```
        for destination, contenders in destination_groups.items():
            if len(contenders) == 1:
                approved[contenders[0]] = destination
            else:
                # Choose one winner; others remain in place.
                winner = random.choice(contenders)
                approved[winner] = destination
```

```
                for agent in contenders:
                    if agent is not winner:
                        approved[agent] = agent.position
```

```
        # Commit all approved positions together.
        # This avoids transient occupancy conflicts.
```

```
        for agent in self.agents:
            self.grid.remove_agents([agent])
```

```
        for agent in self.agents:
            agent.position = approved.get(
                agent, agent.position
            )
            agent.path.append(agent.position)
```

```
        for agent in self.agents:
            self.grid.add_agents([agent])
```

```
# -------- Model Parameters --------
```

```
parameters = {
    "agents": num_agents,
    "grid_size": (grid_size, grid_size),
    "steps": num_steps,
    "min_distance": min_distance
}
```

```
# -------- Run Model --------
```

```
model = RandomWalkModel(parameters)
model.setup()
```

```
# -------- Animation --------
```

```
fig, ax = plt.subplots()
```

```
ax.set_xlim(0, grid_size)
ax.set_ylim(0, grid_size)
ax.set_xticks(range(grid_size + 1))
ax.set_yticks(range(grid_size + 1))
ax.set_aspect("equal")
ax.grid(True)
```

```
scat = ax.scatter([], [], s=100)
```

```
lines = [
    ax.plot([], [], linewidth=1)[0]
    for agent in model.agents
]
```

```
def update(frame):
    model.step()

    x = [agent.position[0] + 0.5 for agent in model.agents]
    y = [agent.position[1] + 0.5 for agent in model.agents]

    scat.set_offsets(list(zip(x, y)))

    for agent, line in zip(model.agents, lines):
        path_x = [p[0] + 0.5 for p in agent.path]
        path_y = [p[1] + 0.5 for p in agent.path]

        line.set_data(path_x, path_y)

    return scat, *lines


ani = animation.FuncAnimation(
    fig,
    update,
    frames=num_steps,
    blit=True,
    repeat=False
)

plt.show()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size? If the number of agents is very high compared to the grid size, a lot of them might overlap positions due to limited space.
2. How does increasing the number of steps affect the **spread of agents**? Increasing the number of steps make the agents more spread out, especially when enforcing rules that make the agents avoid each other.
3. Do the agents ever move outside the grid? Why or why not? They don't due to the simulation fixing their position if they attempt to move outside the grid.
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)? This simulation could be used to explain crowd movement logic in certain animals that exhibit crowding behavior, like elephant herds and schools of fish.